

EIGI FDE Batch 01

Frontend Activity Problem Statement

Session Task: Build a Small Frontend App Using an External API

1. Objective

Your task is to build a small frontend-only application that applies the concepts covered in the frontend session.

This task is not about making a perfect production app. The goal is to understand how a frontend application works end to end:

- how users interact with the UI
- how frontend manages state
- how frontend calls an API
- how API data is displayed
- how loading, error, and empty states work
- how a simple activity can be organized cleanly

You do **not** need to build a backend for this task.

2. What You Need to Build

Build a small web app using any one API from **API Ninjas**.

Your app should:

1. Take user input from the UI.
 2. Call an API Ninjas endpoint using an API key.
 3. Display the API response in a clean frontend interface.
 4. Handle loading, error, and empty states.
 5. Use a basic folder structure.
 6. Be presentable as a small demo.
-

3. Suggested Tech Stack

Use the simplest stack that lets you build and understand the flow.

Recommended

- HTML
- CSS or Tailwind CSS
- JavaScript

Optional, if you already know it

- React
- Vite
- TypeScript

Do not spend too much time setting up advanced tools. The main goal is to understand the frontend flow.

4. API Requirement

Use **API Ninjas** for the activity.

You will need to:

1. Create an account on API Ninjas.
2. Generate an API key.
3. Choose one API endpoint.
4. Call the endpoint from your frontend.
5. Display the response.

API Ninjas APIs generally use the following request header:

```
X-Api-Key: YOUR_API_KEY
```

For this exercise, it is acceptable to use the API key directly in frontend code because this is a learning-only task.

In a real production app, API keys should not be exposed in frontend code. They should be protected using a backend or server-side proxy.

5. Choose Your API

Choose **any one API endpoint** from the API Ninjas API directory.

You are free to choose the API based on what you find interesting, as long as the API allows you to build a small frontend flow with:

- user input or button action
- API request
- loading state
- error state
- result display
- clean UI

Before building, open the API documentation and understand:

1. What endpoint URL should be called?
2. What query parameters are required?

3. What request headers are needed?
4. What does the response look like?
5. Which fields will you display in the UI?
6. What can go wrong?

Examples of APIs you may explore:

- Weather API
- Facts API
- Quotes API
- Nutrition API
- Domain API
- City API
- Dictionary API
- Exercise API
- Recipe API
- Any other suitable API from API Ninjas

Do not choose an API blindly. Choose one that you can explain and display clearly.

Your activity idea should be simple:

```
User action -> API request -> response data -> UI update
```

Examples:

```
User enters a city -> app shows weather details
User clicks a button -> app shows a random quote
User enters a food item -> app shows nutrition data
User enters a word -> app shows dictionary meaning
User enters a domain -> app shows domain information
```

6. Minimum Requirements

Your activity must include the following:

UI Requirements

- A clear title
- At least one input or button
- A result display section
- A loading indicator
- An error message area
- Basic responsive layout

Functional Requirements

- Call at least one API Ninjas endpoint
- Use the API key correctly
- Display real API response data
- Handle empty input if your app uses input
- Handle API failure gracefully
- Avoid breaking the UI if the response is missing some fields

Code Requirements

- Keep code readable
- Use meaningful variable names
- Separate API call logic if possible
- Avoid unnecessary copy-paste
- Add basic comments where helpful

7. Suggested Folder Structure

For a simple HTML/CSS/JavaScript activity:

```
frontend-activity/  
  index.html  
  styles.css  
  script.js  
  README.md
```

For a slightly better organized JavaScript activity:

```
frontend-activity/  
  index.html  
  src/  
    api.js  
    main.js  
    ui.js  
  styles/  
    styles.css  
  README.md
```

For a React activity:

```
frontend-activity/  
  src/  
    app/  
      App.jsx  
    features/  
      search/  
        SearchForm.jsx
```

```
    ResultCard.jsx
    api.js
  shared/
    components/
      Loader.jsx
      ErrorMessage.jsx
  main.jsx
  README.md
```

Use the structure that matches your current skill level. Do not over-engineer the activity.

8. Frontend Concepts You Should Apply

Your activity should demonstrate these concepts from the session:

User Intent

The user clicks a button or enters a query.

Example:

```
User enters city name → clicks Search
```

UI State

The app should know what is happening right now.

Examples:

```
loading = true
error = "Something went wrong"
result = API response data
```

API Calling

Your app should send a request to API Ninjas and receive a response.

Example:

```
Frontend → API Ninjas → Response → UI update
```

Feedback

The user should always understand what is happening.

Examples:

- Show loading while request is running
- Show result after success
- Show error if request fails
- Show message if input is empty

Clean UI

The app does not need to be fancy, but it should be usable.

Focus on:

- spacing
 - readable text
 - clear button
 - visible result area
 - mobile-friendly layout
-

9. Example User Flow

For a Weather App:

1. User opens the app.
 2. User sees a title and city input field.
 3. User enters **Nagpur**.
 4. User clicks **Search**.
 5. App shows loading state.
 6. App calls API Ninjas Weather API.
 7. API returns weather data.
 8. App displays weather information.
 9. If the request fails, app shows an error message.
-

10. Evaluation Rubric

Your activity will be evaluated based on the following criteria.

1. Working API Integration

The app should successfully call an API and display real response data.

2. UI Clarity

The interface should be understandable and usable.

3. State Handling

The app should handle loading, error, empty, and success states.

4. Code Organization

Files should be named properly and organized clearly.

5. Understanding

You should be able to explain how your app works.

6. Demo Quality

You should be able to run the app and show the flow without confusion.

11. Demo Questions

Be ready to answer these questions during review:

1. What did you build?
 2. Which API did you use?
 3. Where is the API call happening in your code?
 4. How are you handling loading state?
 5. How are you handling errors?
 6. What happens if the user gives empty input?
 7. How did you structure your files?
 8. What would you improve if you had more time?
-

12. Submission Requirements

Submit the following:

1. Activity folder
2. Short README file
3. Screenshot of the working app
4. Brief explanation of what you built

Your README should include:

- Activity name
 - What the app does
 - API used
 - How to run it locally
 - What you learned
-

13. Time Box

Recommended time:

60 to 90 minutes

Suggested split:

- 10 minutes: choose API and understand response
- 15 minutes: create UI
- 20 minutes: connect API
- 15 minutes: handle states
- 10 minutes: polish UI
- 10 minutes: prepare demo and README

14. Important Notes

- Do not try to build a huge activity.
- Do not add backend.
- Do not use too many libraries.
- Do not focus only on design and forget functionality.
- Do not ignore loading and error states.
- Do not copy code blindly without understanding it.

The goal is not complexity.

The goal is to understand this flow:

User action → frontend state → API request → response → UI update

15. Expected Outcome

By the end of this task, you should understand:

- how frontend captures user input
- how frontend calls external APIs
- how API responses are displayed
- how loading and error states improve user experience
- how small frontend activities should be structured
- why frontend is more than just visual design

16. Final Reminder

Build something small, working, and explainable.

A simple app that works clearly is better than a complicated app that nobody can understand.